

Paradigmas de Programação — Aula 08:

Concorrência, gorrotinas e canais

Concorrência e paralelismo

Objetivos da aula

- Distinguir **concorrência** de **paralelismo**
- Explicar a **condição de corrida** em threads com memória compartilhada
- Escrever programas Go com **gorrotinas** e **canais**
- Medir o ganho da concorrência em tarefas que **esperam**

Duas propriedades diferentes

- **Concorrência:** propriedade da **estrutura** do programa — tarefas em andamento no mesmo intervalo de tempo, sem que uma precise terminar para a outra começar
- **Paralelismo:** propriedade da **execução** — tarefas executando no mesmo instante, em núcleos distintos
- Concorrência é *lidar* com muitas coisas ao mesmo tempo; paralelismo é *fazer* muitas coisas ao mesmo tempo (PIKE, 2012)

A mesma estrutura, duas execuções

Concorrência em um núcleo (intercalação)

núcleo 1: [A][B][A][B][A][B]

Paralelismo em dois núcleos

núcleo 1: [A][A][A]

núcleo 2: [B][B][B]

- O programa pode ser o mesmo nos dois casos
- A estrutura concorrente é escrita pelo programador; o paralelismo depende dos núcleos disponíveis na execução
- Estrutura concorrente **permite** paralelismo, mas não o garante

Uma não implica a outra

	Sem paralelismo	Com paralelismo
Sem concorrência	programa sequencial	mesma operação sobre um vetor, em vários núcleos
Com concorrência	tarefas intercaladas em um núcleo	tarefas distribuídas entre núcleos

Tarefa que espera × tarefa que calcula

- **Limitada por E/S (*I/O-bound*)**: passa o tempo esperando rede, disco, usuário
 - concorrência reduz o tempo **mesmo em um núcleo**: a espera de uma é aproveitada por outra
 - três consultas de 100, 250 e 180 ms: **530 ms** em sequência, **250 ms** concorrentes
- **Limitada por CPU (*CPU-bound*)**: passa o tempo calculando
 - em um núcleo, intercalar não ganha nada
 - o ganho só vem com paralelismo, limitado ao número de núcleos